

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF09:15:Sockets & Ports (& Services)

1. Ausgangspunkt

Wie haben ausführlich besprochen [→ **ZP:Sheet2**]:

- Die Aktivitäten, die nötig sind, um eine Nachricht über Broadcastgrenzen hinweg zu senden
 - Sender (BCA/IP):
 - * Feststellen, ob Ziel-IP in eigener Broadcastdomain.
 - * Wenn nicht, ARP-Request bzgl. Router-MAC-Adresse
 - * Nachricht an Ziel-IP und Router-MAC als Weiterleitungsauftrag.
 - Router:
 - * Feststellen, ob Ziel-IP in BCB.
 - * Wenn ja, ARP-Request bzgl. MAC-Adresse des Ziel-IP-Rechners.
 - * Nachricht umformulieren.
 - * Nachricht an Ziel-IP mit Router-MAC und BCA-IP.
 - Empfänger: verstehen.
- Die Abfolge der dafür nötigen Zwischennachrichten [→ **ZP:Sheet3**]
- Den Unterschied zwischen ARP-Package und IP-Package [→ **ZP:Sheet4**]
 - den wir in der Dokumentation ‘vernachlässigt’ haben.
- Das Package-in-Package-Prinzip [→ **ZP:Sheet5**]

ABER

- Es reden *nicht Rechner mit Rechner*,
- sondern ****(Client-)_App**** mit ****(Server-)_App****!

Wie geht das? [→ **ZP:Sheet6**]

ÜBUNG LF09/15:services:01

- 1) Bitte wählen Sie
 - 2 Mitschülerinnen, die je eine Routerhälfte mit eingebettetem Switch simulieren
 - 2 Mitschülerinnen, die je ein Betriebssystem simulieren
 - 2 Mitschülerinnen, die je eine Netzwerkkarte simulieren
 - 2 Mitschülerinnen, die je eine App simulieren
- 2) Wählen Sie ein oder zwei Protokollantinnen
- 3) (Er)finden Sie einen allgemeingültigen, mehrfach parallel ausführbaren Prozess,
 - der die bisher besprochenen Techniken nutzt,
 - der damit eine Nachricht von APP(BCA) zu APP(BCB) und die Antwort zurück übermittelt und
 - der die Regel beachtet, dass nur das Betriebssystem von der Netzwerkkarte lesen und auf die Netzwerkkarte schreiben kann.
- 4) Führen Sie Ihren Prozess als Schreibtischtest durch.
- 5) Dokumentieren Sie Ihren Prozess als Aktivitätsdiagramm.

Lösung: Request ([→ **ZP:Sheet7**]) Response ([→ **ZP:Sheet8**])

2. Die Kommunikation zwischen (Client)App und (Server)App

bedarf auf

- **Serverseite:**
 - **einer Akkreditierungsphase:**
 - * ServApp beauftragt ServOps, ihm über einen Socket alle eingehenden Nachrichten zu einem Port zu übermitteln.
 - * ServApp beauftragt ServOps, alle Nachrichten, die die Server-App auf den Socket schreibt, angemessen über das Netzwerkinterface zu versenden.
 - * ServOps gibt einen (an den Port gebundenen) Socket mit IO-Rechten für die Server-App zurück.
 - **einer Lausch-und Annahmephase:**
 - * ServOps lauscht auf Netzwerkinterface.

- * ServOps analysiert eingehenden Nachrichten.
- * Geht eine Nachricht für den akkreditierten Port ein, schreibt ServOps sie auf den akkreditierten Socket.
- * ServApp lauscht auf akkreditierten Socket.
- * Geht eine Nachricht darauf, liest ServApp sie vom Socket und interpretiert sie.

– **einer Schreibphase:**

- * ServApp schreibt Nachricht auf den Socket [mit Destination-IP-Adresse (= Source-IP-Adresse aus dem Request) und Destination-Port (= Source-Port aus dem Request)].
- * ServOps lauscht auf den akkreditierten Socket.
- * ServOps liest Nachricht vom akkreditierten Socket.
- * ServOps komplettiert Nachricht mit Source-IP-Adresse und Source-Port (wie in der Akkreditierungsphase angestimmt).
- * ServOps versendet Nachricht mit ARP+MSG an Router.

• **Clientseite:**

– **einer Akkreditierungsphase:**

- * ClientApp beauftragt OPS, ihm einen an einen Interimsport gebundenen Socket für eine Nachricht zu übergeben.
- * ClientApp schreibt Nachricht auf den Socket [mit Destination-IP-Adresse (bekannt) und Destination-Port (= erhaltener Interimsport) auf den Socket.
- * ClientOps lauscht auf den akkreditierten Socket.
- * ClientOps liest Nachricht vom akkreditierten Socket.
- * ClientOps komplettiert Nachricht mit Source-IP-Adresse (wie in der Akkreditierungsphase angestimmt).
- * ClientOps versendet Nachricht mit ARP+MSG an Router.

Anmerkung:

Wer genau welche Source-IP und Source-Port in das Paket einfügt, hängt von der Implementierung der IP-Bibliothek und dem Status Client bzw. Server ab

3. Die Verpackungssystematik [→ ZP:Sheet9]

Welche Protokolle im Payload eines Packages zulässig sind, gibt jeweils ein Kenner an. Das Kennungsfeld heißt

- im Ethernetpackage: **Ethertype**. Zu möglichen Werten, siehe:
 - <https://de.wikipedia.org/wiki/Datenframe>

- <https://de.wikipedia.org/wiki/Ethernet>
- im IPv4-Package: **Protokoll**. Zu möglichen Werten, siehe:
 - <https://de.wikipedia.org/wiki/IPv4>
- im IPv6-Package: **Next Header**. Zu möglichen Werten, siehe:
 - <https://de.wikipedia.org/wiki/IPv6>

Zwischenfrage:

Im TCP-Package und UDP-Package gibt es kein Feld, das den Typ des Payloads bestimmt. Warum nicht?

- im TCP-Package: **Destination Port**. Zu möglichen Werten, siehe:
 - https://de.wikipedia.org/wiki/Transmission_Control_Protocol
- im UDP-Package: **Destination Port**. Zu möglichen Werten, siehe:
 - https://de.wikipedia.org/wiki/User_Datagram_Protocol

4. Die Systematik der Portnummern [→ ZP:Sheet10]

- (Layer-IV)-Port := Zahl zwischen 0 und 65535
- **Systemport** := 0 - 1023 (auch 'well-known ports', weil registriert bei der IANA). Beispiel:
 - 7 := ECHO-Service
 - 20 := FTP (Datenübertragung: nur TCP)
 - 21 := FTP (Verbindungsaufbau und Steuerung: TCP, UDP)
 - * dient der Übertragung von Dateien
 - * nutzt einen Kanal zur Übertragung von serverseitig ausgeführten Befehlen (cp, mkdir, rename, ...)
 - * nutzt den anderen Kanal zur Date(ie)nübertragung
 - * beides eher vorsichtig nutzen, Passwörter / Datenübertragung im Klartext
 - * Alternativen sind [sftp](#) oder [scp](#) (aus ssh-Pakete)
 - 22 := SSH (TCP, UDP)
 - * zur verschlüsselten Übertragung von Befehlen, die serverseitig in einer Shell ausgeführt werden
 - 23 := Telnet (unverschlüsseltes Textprotokoll, z. B. für Fernwartung)
 - * zur unverschlüsselten Übertragung von Befehlen, die serverseitig in einer Shell ausgeführt werden

- * beides eher vorsichtig nutzen, Passwörter / Datenübertragung im Klartext
 - 25 := SMTP (Simple Mail Transfer Protocol) /* Kommunikation unter Mailservern */
 - * dient zur Kommunikation von Mailservern untereinander
 - * 110 :- pop3 /* Kommunikation Mailclient zu Mailserver */
 - geholte Mails werden direkt auf dem Server gelöscht, deshalb keine Synchronisation über mehrere Geräte möglich
 - * 143 (993) :- imap(s) /* Kommunikation Mailclient zu Mailserver */
 - Mails werden serverseitig nur gelöscht, wenn explizit angestoßen. Deshalb implizit Synchronisation über mehrere Geräte
 - 67 := Bootstrap Protocol (auch von DHCP genutzt: nur per UDP)
 - 80 := HTTP (TCP, UDP)
 - * dient praktisch zu Download von Date(ie)n
 - * bietet als Protokoll aber einen Set von Befehlen 'GET, POST, HEAD, PUT, PATCH, DELETE, ...
 - * die serverseitig wirksamen Client-Befehle PUT, PATCH, DELETE, ... sind 'gefährlich', müssen i.d.R. gesondert aktiviert werden und verlangen ein spezielles Rechtemanagement
 - 123:= NTP (Network Time Protocol)
 - * dient der Synchronisation von Rechnern
 - * von zentraler Uhr wird die gerade gültige Uhrzeit über einen Stack von Servern verteilt.
 - * Übertragungszeit verhindert eine wirklich genaue Synchronisation.
 - * Große Zeitunterschiede werden bei der Übernahme in kleineren Schritten ausgeglichen.
 - * → https://de.wikipedia.org/wiki/Network_Time_Protocol
 - 319 := PTP (Precision Time Protocol) / Events (nur UDP)
 - 320 := PTP (Precision Time Protocol) / General Messages (nur UDP)
 - * dient der genauen Synchronisation von Rechnern eines engeren Netzes
 - * bei der Übermittlung der Zeit wird mittels Antworten der Übertragungsdelay einkalkuliert
 - * → https://de.wikipedia.org/wiki/Precision_Time_Protocol
 - 443 := HTTPS (Hypertext Transfer Protocol over SSL/TLS: TCP, UDP)
 - * wie [http](http://), nur mit 'eingebauter' Verschlüsselung
- **Registrierte Ports** := 1024 – 49151 (irgendwo registriert oder kraft Konvention / 'Gebrauchsrecht' festgelegt). Beispiel:
 - 1434 := MSSQL

- 1883 := MQTT
- 3306 := Mysql
- 8080 := HTTP Alternative, für Proxyserver (Vermittler zwischen Client und realem Server) genutzt

- **Dynamische Ports** := 49152 – 65535 (Ports für Clients und Server-Clone)

vgl. [https://de.wikipedia.org/wiki/Liste_der_Portnummern]

5. Die wichtigsten Services im Überblick [→ ZP:Sheet11]

6. Package-in-Package-Übung

Bauanleitung:

Die Lehrerin ...

1. organisiert vorab 4 * 3 ineinander legbare Kartons (Matruschka-Prinzip),
2. druckt dann die Datei `preambels.odt` mit Libre-Office aus,
3. legt danach - entsprechend der Lösungsbeschreibung - in jedes innerste Paket die Messages von den Seiten P1 - P4,
4. schneidet schließlich für jedes Paket die entsprechenden Präambelzeilen ab,
5. klebt die Präambeln dann - von innen nach außen gehend - je an die Unterseite der einzelnen Schachteln und
6. verpackt zuletzt alles zusammen in Geschenkpapier (Symbol für Ethernetpackage)

ÜBUNG LF09/15:services:02

- ☐ Bitte teilen Sie sich in vier Gruppen auf.
- ☐ Bitte nehmen Sie gruppenweise Ihr 'Geschenk' entgegen und entpacken Sie es vorsichtig.
- ☐ Kriegen Sie bitte heraus, was Sie da bekommen haben. (ca. 40 Min).
- ☐ Erstellen Sie eine kurze Präsentation, anhand derer Sie Ihren Mitschülerinnen charmant erläutern können, was Sie da bekommen haben (ca. 20 Min.). Leitfragen dazu sind:
 - Was haben Sie bekommen?
 - Wie ist es aufgebaut und wie funktioniert es?
 - Wofür stehen die Komponenten des 'Geschenks' in der Wirklichkeit?
- Präsentieren Sie Ihr Ergebnis (Pkg 1, 2, 3, 4)

Lösung:

P1 echo (tcp/ipv4)

- L7:

```
1 Well done
```

- L4 (tcp): 0xFFFB 0x0007 4x 4x 0x0 3x 4x
- L3 (ip): 4x 4x 1x 0x06 2x 0x23451234 0x43215423>
- L2 (ef): 8x 0x847321 0x123748 4x 0x0800 4? 12x

Besonderheit:

- Echo-Server-Port = 7 = 0x07,
- Protocol-Nummer für TCP = 6 = 0x06
- EType für IP = 0x0800
- Geschenkpapier ist Ethernetpackage.

P2 http (tcp/ipv4)

- L7:

```
1 GET /index.html HTTP/1.1\n
2 Host: http://tierschutz.hessen.de/\n
3 \n
```

- L4 (tcp): 0xFFFA 0x0050 4*x 4*x 0x0 3*x 4*x
- L3 (ip): 4x 4x 1x 0x06 2x 0x23451234 0x43215423
- L2 (ef): 8x 0x847321 0x123748 4x 0x0800 4? 12x

Besonderheit:

- Http-Server-Port = 80 = 0x50
- Protocol-Nummer für TCP = 6 = 0x06
- EType für IP = 0x0800
- Geschenkpapier ist Ethernetpackage.

P3 http (udp/ipv4)

- L7:

```
1 GET /index.html HTTP/3.0\n
2 Host: http://vegan.de/\n
3 \n
```

- L4 (udp): `0xFFFC 0x0050 0x0030 2*x`
- L3 (ip): `4x 4x 1x 0x11 2x 0x23451234 0x43215423`
- L2 (ef): `8x 0x847321 0x123748 4x 0x0800 4? 12a`
- Geschenkpapier ist Ethernetpackage.

Besonderheit:

- Http-Server-Port = 80 = 0x50
- Protocol-Nummer für UDP = 17 = 0x11
- EType für IP = 0x0800
- Geschenkpapier ist Ethernetpackage.

P4 telnet (tcp/ipv6)

- L7: > "echo 'well done' "
- L4 (tcp): `0xFFFD 0x0017 4*x 4*x 0x0 0x? 3*x 4*x`
- L3 (ipv6): `4x 0x10 0x06 1x 0x2001234767891618 0x2001816198767432`
- L2 (ef): `8x 0x847321 0x123748 4x 0x86DD 4? 12`

Besonderheit:

- Telnet-Server-Port = 23 = 0x17
- Protocol-Nummer für TCP = 6 = 0x06
- EType für IPv6 = 0x86DD
- Geschenkpapier ist Ethernetpackage.

7. Zur Analyse von 'listening Ports'

ÜBUNG LF09/15:services:03

- ☐ Lassen Sie sich über die folgenden Befehle die auf Ihrem Rechner arbeitenden Server-Applikationen und ihre Ports anzeigen:
 - LNX/MACOS: `netstat -ano | grep tcp`
 - LNX/MACOS: `netstat -ano | grep udp`
 - LNX/MACOS: `lsof -i -n | grep tcp`
 - W11/pwsh (V.7.x): `netstat -ano | select-string TCP`
 - W11/powershell (V.5.1): `netstat -ano | findstr HERGESTELLT`
- ☐ Führen Sie diese Befehle nun als Root/Admin aus (LNX: `sudo lsof -i -n`). Beschreiben Sie, was jetzt angezeigt wird.?

- ☐ Beschreiben Sie, was auf Ihrem Rechner 'los ist'.

Lösung:

- mit `netstat -ano | grep tcp`
 - listening auf Loopback-IPv4-Adressen
 - * IPv4-Server: DNS(53), MySQL(3306), MQTT(1883), IPP(631), SSH(22), SMB/NetBIOS(139,445)
 - * IPv6-Server: MQTT(1883), IPP(631), http(80), SSH(22), SMB/NetBIOS(139,445)
 - established IPv6: 2003:c0:bf2e:8ba3:44104 2606:50c0:8003::154:443 (http)
- mit `lsof -i -n`

```
1 codium 53309 IPv6 TCP [2003:c0:bf2e:8ba3:afbb:939d:4459:6ae1
  ]:44104->[2606:50c0:8003::154]:https (ESTABLISHED)
2 chrome 57450 IPv6 TCP [2003:c0:bf2e:8ba3:afbb:939d:4459:6ae1
  ]:54632->[2a00:1450:400c:c06::bc]:5228 (ESTABLISHED)
3 chrome 57450 IPv4 TCP 192.168.2.102:59790->18.245.60.112:https (
  ESTABLISHED)
4 chrome 57450 IPv6 TCP [2003:c0:bf2e:8ba3:afbb:939d:4459:6ae1
  ]:54198->[2a00:1450:4001:c13::8b]:https (ESTABLISHED)
5 chrome 57450 IPv4 TCP 192.168.2.102:53658->107.23.108.68:https (
  ESTABLISHED)
6 chrome 57450 IPv4 TCP 192.168.2.102:46040->18.233.64.208:https (
  ESTABLISHED)
7 chrome 57450 IPv4 TCP 192.168.2.102:44446->100.51.137.153:https (
  ESTABLISHED)
```

=>

- Codium hat Verbindung nach außen offen.
- Browser hat bidirektionale Verbindung mit Google offen (Port 5228)
- Browser hat vier https / ipv4 Verbindungen offen
- Browser hat eine https / ipv6 Verbindung offen

8. Übung zur Client-Server-Programmierung

ÜBUNG LF09/15:services:04

- ☐ Programmieren Sie in Python einen TCP/IP-Echo-Server (mit Pythonmodul `socket`), der
 - ☐ auf der Konsole mit `python3 echo-server.py` gestartet wird,

- ☐ hintereinander mehrere Requests an die Loopbackadresse entgegennimmt,
 - ☐ sich für einen festen Port aus dem Bereich der dynamischen Ports zuständig fühlt,
 - ☐ anzeigt, wann er zu beantworten bereit ist,
 - ☐ die Daten des anfragenden Clients anzeigt,
 - ☐ den Erfolg einer Rückmeldung anzeigt.
- ☐ Programmieren Sie in Python einen Echo-Client (mit Pythonmodul `socket`), der
- ☐ auf der Konsole mit `python3 echo-client.py` gestartet wird,
 - ☐ nach Erhalt des Sockets konfigurierbar lange abwartet, bevor er
 - ☐ einen Echo-Request an die Loopbackadresse absetzt,
 - ☐ dabei den oben festgelegten Port als Zielport benutzt,
 - ☐ den Request vor Absenden ausgibt,
 - ☐ sein Warten auf die Antwort signalisiert,
 - ☐ die eingegangene Antwort ausgibt.
- ☐ Testen Sie den Service über zwei Konsolenfenster.
- → <https://docs.python.org/3/library/socket.html>
 - → https://www.w3schools.com/python/ref_module_socket.asp

Lösung: Echo-Client: [→ **ZP:Sheet12**], Echo-Server [→ **ZP:Sheet13**]

9. Übung zur Server-Status-Überwachung

ÜBUNG LF09/15:services:05

- ☐ Öffnen Sie 3 Terminals nebeneinander.
- ☐ Starten Sie im ersten Terminal Ihren Echo-Server mit `python3 echo-server.py`.
- ☐ Starten Sie im zweiten Terminal Ihren Echo-Server mit `python3 echoclient.py`.
- ☐ Rufen Sie im dritten Terminal (fortlaufend) vor jeder Ausgabe in Terminal 2 `lsof -i -n`.
- ☐ Beschreiben Sie, wie sich der Status Ihres Echo-Servers immer wieder verändert.

Lösung:

- Server LISTENING
 - Server ESTABLISHED (direkt, wenn Client seinen Socket zum Schreiben bekommen hat.)
 - ...
-

10. UDP versus TCP/IP

• UDP:

- steht für **User Datagram Protocol**
- ist ein "verbindungsloses, nicht-zuverlässiges" Protokoll
 - * keine Garantie, dass ein einmal gesendetes Paket auch ankommt
 - * keine Garantie, dass die Pakete in derselben Reihenfolge ankommen, in der sie gesendet wurden
 - * keine Garantie, dass Pakete nur einmal beim Empfänger eintrifft.
- UDP-Package wird ,von der Client-/Server-App - beginnend mit Layer VII bzw. V - zusammengestellt.
- Direkt über die Layer IV, III, II und I ineinander geschachtelt verpackt (Matruschka-Prinzip)
- dann versendet,
- wieder empfangen und entpackt
- und über den Socket an die App hochgereicht.
- Rekonstruktion der gesendeten Nachricht mittels Sortierung und Komplettierung mehrerer UDP-Pakete ist Aufgabe der APP
- entspricht 'Fire-and-Forget'
- → https://de.wikipedia.org/wiki/User_Datagram_Protocol

• TCP

- steht für **Transmission Control Protocol**
- eine Ende-zu-Ende-Verbindung im (Voll-|Halb-)duplex-Mode, definiert durch zwei Endpunkte, die jeweils aus einem geordneten Paar von IP-Adresse und Port bestehen:
 - * [**<Lokaler Rechner, Lokaler Port>**, **<Entfernter Rechner, Entfernter Port>**]
- verwaltet von der Transportschicht, die der empfangenden App gegenüber garantiert (oder ein Zusammenbruch der Verbindung an die App meldet), dass
 - * gesendete Pakete auch ankommen,
 - * die Pakete in derselben Reihenfolge ankommen, in der sie gesendet wurden,
 - * dass Pakete nur einmal eintreffen.
- TCP-Package wird von der Client-/Server-App - beginnend mit Layer VII bzw. V - zusammengestellt.
- Auf Layer-IV kommen Sortierinformationen etc. hinzu.
- Dann wird es über die Layer IV, III, II und I ineinander geschachtelt verpackt (Matruschka-Prinzip),
- dann versendet,

- wieder empfangen und entpackt.
- Auf Layer IV wird die Reihenfolge der Pakete rekonstruiert (Sortierung, Nachforderung)
- bis die Pakete - in der gesendeten Reihenfolge - über den Socket an die App hochgereicht werden.
- Rekonstruktion der gesendeten Nachricht mittels Sortierung und Komplettierung mehrerer TCP-Pakete wird vor der APP verborgen.
- → https://de.wikipedia.org/wiki/Transmission_Control_Protocol
- → https://en.wikipedia.org/wiki/Transmission_Control_Protocol

Anmerkungen zu TCP/IP:

Befund:

- Abstrakt gesehen kann derselbe Server (Programminstanz) mit demselben Port verschiedene Clients bedienen:
 - Verbindung A: 192.168.110.1:80 <-> 192.168.110.1:65110,
 - Verbindung B: 192.168.110.1:80 <-> 192.168.110.21:65111.
- Auf demselben Rechner können aber nicht mehrere Instanzen desselben Programms über denselben Port/Socket agieren.

Konsequenz in Sachen Performance:

- strikte serielle Abarbeitungen von Requests seitens des Servers.
- langsam!
- Lösungen:
 - **A)** Aufsetzen einer mittels eines Loadbalancers zusammengefassten Serverfarm (Prinzip: Apothekenschlange mit Wartebereich an der Tür.)
 - **B)** Thread basierte Serverprogramme = Starten der Server-App in mehreren Threads
 - * Elternthread unter root gestartet mit Bindung an Port 80 / 443
 - * Child-Threads per fork 'nachgeladen' von Server-App
 - * Alle (Child-)Threads nutzen dasselbe Codesegment und dasselbe Datensegment, aber haben je eigene Befehlszähler und Stagesegmente
 - * Elternthread liest Request vom Socket, gibt in an einen (Child-)Thread zu Bearbeitung (Elternthread ist Request-Dispatcher)
 - * Nach Berechnung der Response schreibt (Child-)Thread die Antwort auf den Socket (Überschreibenschutz per Semaphoren)
 - * Das ist gewissermaßen eine Rechner interne Apothekenschlange.
 - * → [https://de.wikipedia.org/wiki/Thread_\(Informatik\)](https://de.wikipedia.org/wiki/Thread_(Informatik))

11. von 'LISTENING' zu 'ESTABLISHED' TCP/IP

Frage: Was passiert zwischen dem Serverstatus 'LISTENING' und dem Serverstatus 'ESTABLISHED'?

Antwort: Initialisierung / Aufbau der stabilen Duplex-Verbindung mit 3 Wege-Handshake:

- (A) Client: sendet Server ein SYN-Paket:
 - setzt SYN-Bit
 - belegt Feld *Sequence Number* im TCP-Header mit Wert x
- (B) Server: antwortet mit:
 - TCP-RST falls keine Verbindung aufgebaut werden kann (kein Service zu dem Port aktiv).
 - mit SYN/ACK-Paket als Bestätigung (falls Port offen = für Serviceanfragen nutzbar)
 - * setzt ACC-Bit
 - * belegt Feld *Acknowledgement Number* mit x+1
 - * belegt Feld *Sequence Number* mit y
- (C) Client: sendet Server ein ACK-Paket mit seq+1 und acc=y+1

Abbau entsprechend

Die TCP-Sockets auf Client- und Serverseite können dabei prinzipiell verschiedene Zustände annehmen:

- LISTEN [Server] : auf Request warten
- SYN-SENT [Client] : auf Bestätigung des Aufbaus warten
- SYN-RECEIVED [Server] : auf die Bestätigung des Clients warten
- ESTABLISHED [Server, Client] : Duplex-Verbindung existiert
- FIN-WAIT-1 [Server, Client] : warten auf: a) Termination-Request der Gegenseite oder b) Bestätigung eines zuvor gesendeten Termination-Requests
- FIN-WAIT-2 [Server, Client] : warten auf Termination-Request der Gegenseite
- CLOSE-WAIT [Server, Client] : auf Termination-Request des lokalen Users warten.
- CLOSING [Server, Client] : auf Bestätigung des Termination-Requests durch die Gegenseite warten.
- LAST-ACK [Server, Client] : auf Bestätigung des vorher gesendeten Termination-Requests (mit impliziter eigener Bestätigung) durch die Gegenseite warten
- TIME-WAIT [Server, Client] : die Zeit abwarten, bis alle möglicherweise noch umlaufenden Pakete abgelaufen sind
- CLOSED [Server, Client] : Keine Verbindung mehr

Zeitbegriffe:

- **TTL** (Time To Live) Byte *im IP-Paket*:
 - Wird bei jedem Hop um eins vermindert.
 - Paket wird bei 0 verworfen.
 - Initialwerte sind pro Betriebssystem verschieden: LNX 64, WIN 128, sonst 255
- **MSL** (Maximum Segment Lifetime) : definiert als 2 Minuten, heute oft al 30 sec gesetzt.
 - TIME-WAIT dauert - wegen hin und her - ‘maximal’ (noch) 4 bzw. 1 Minute
- → https://en.wikipedia.org/wiki/Transmission_Control_Protocol
- → https://de.wikipedia.org/wiki/Transmission_Control_Protocol
- → https://de.wikipedia.org/wiki/Time_to_Live
- → https://en.wikipedia.org/wiki/Time_to_live
- → https://en.wikipedia.org/wiki/Maximum_segment_lifetime
- → <https://de.wikipedia.org/wiki/Drei-Wege-Handschlag>